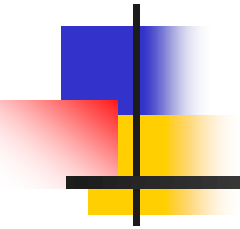


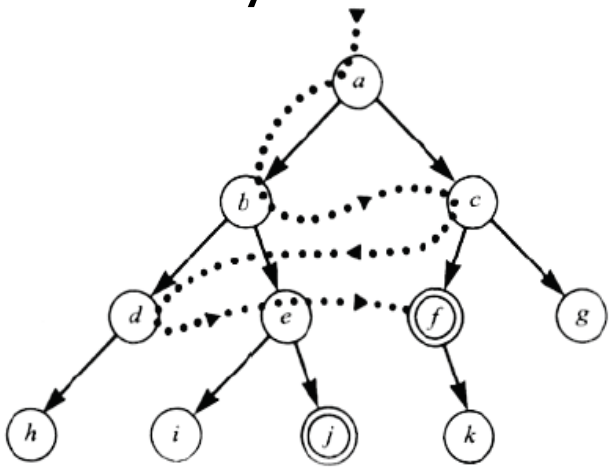
Informed (Heuristic) Search Strategies

Using problem specific knowledge to aid searching

- Without incorporating knowledge into searching, one can have no *bias* (i.e. a preference) on the search space.

- Without a bias, one is forced to look everywhere to find the answer. Hence, the complexity of uninformed search is intractable.

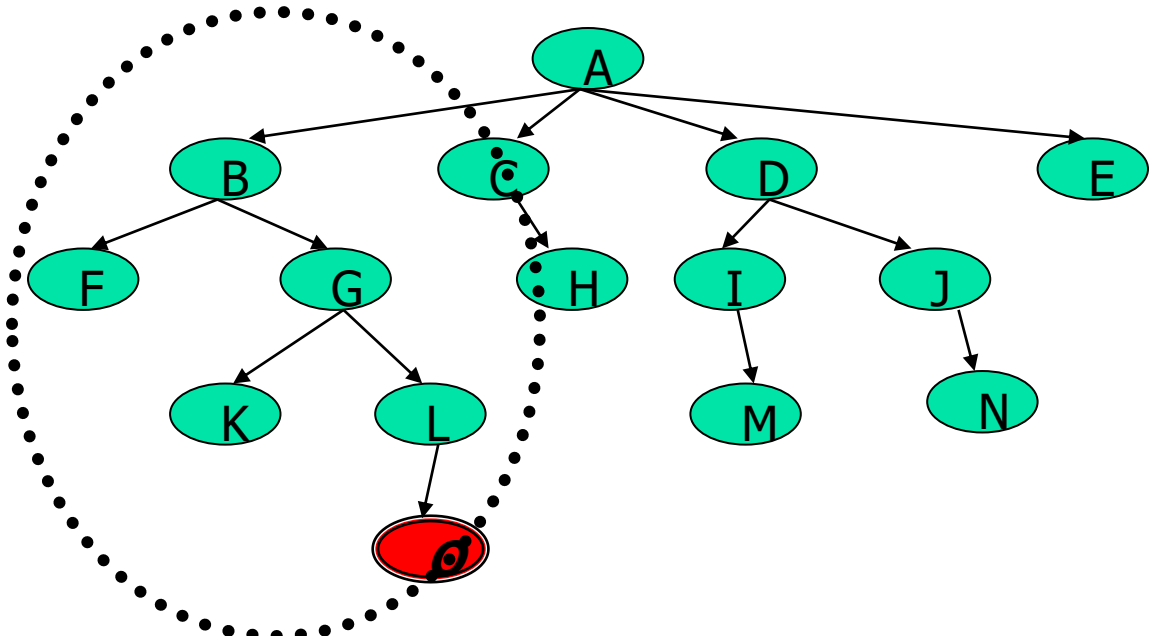
Search everywhere!!



Using problem specific knowledge to aid searching

- With knowledge, one can search the state space as if he was given "hints" when exploring a maze.
  - Heuristic information in search = Hints
- Leads to dramatic speed up in efficiency.

Search only in this subtree!!



More formally, why heuristic functions work?

- In any search problem where there are at most *b* choices at each node and a depth of *d* at the goal node, a naive search algorithm would have to, in the worst case, search around $O(b^d)$ nodes before finding a solution (Exponential Time Complexity).

- Heuristics improve the efficiency of search algorithms by reducing the effective branching factor from *b* to (ideally) a low constant b* such that
  - 1 =< b* << b

Heuristic Functions

- A heuristic function is a function *f(n)* that gives an estimation on the "cost" of getting from node *n* to the goal state – so that the node with the least cost among all possible choices can be selected for expansion first.

- Three approaches to defining *f*:

  - *f* measures the value of the current state (its "goodness")

  - *f* measures the estimated cost of getting to the goal from the current state:
    - *f(n)* = *h(n)* where *h(n)* = an estimate of the cost to get from *n* to a goal

  - *f* measures the estimated cost of getting to the goal state from the *current state* and the cost of the existing path to it. Often, in this case, we decompose *f*:
    - *f(n)* = *g(n)* + *h(n)* where *g(n)* = the cost to get to *n* (from initial state)

Approach 1: f Measures the Value of the Current State

- Usually the case when solving optimization problems
  - Finding a state such that the value of the metric f is optimized
- Often, in these cases, f could be a weighted sum of a set of component values:
  - N-Queens
    - Example: the number of queens under attack …
  - Data mining
    - Example: the "predictive-ness" (a.k.a. accuracy) of a rule discovered

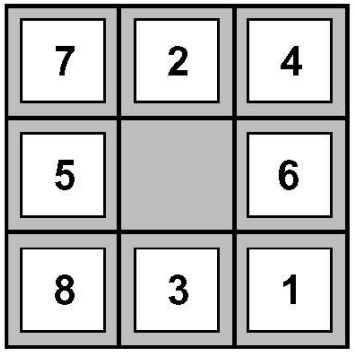
Approach 2: f Measures the Cost to the Goal

A state X would be better than a state Y if the estimated cost of getting from X to the goal is lower than that of Y – because X would be closer to the goal than Y

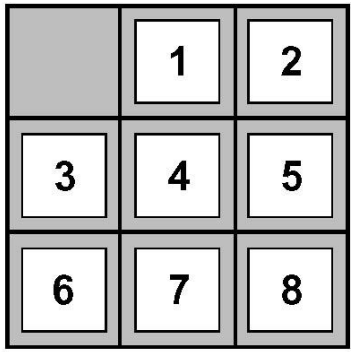
- 8–Puzzle

h_1: The number of misplaced tiles (squares with number).

h_2: The sum of the distances of the tiles from their goal positions.



Start State



Goal State

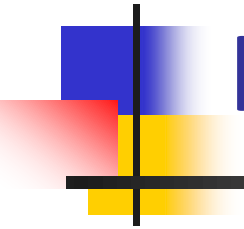
Approach 3: *f* measures the total cost of the solution path (Admissible Heuristic Functions)

- A heuristic function $f(n) = g(n) + h(n)$ is admissible if $h(n)$ **never** overestimates the cost to reach the goal.
  - Admissible heuristics are "optimistic": "the cost is not that much …"
- However, $g(n)$ is the exact cost to reach node n from the initial state.
- Therefore, $f(n)$ never over-estimate the true cost to reach the goal state through node n.
- Theorem: A search is optimal if $h(n)$ is admissible.
  - I.e. The search using $h(n)$ returns an optimal solution.
- Given $h_2(n) > h_1(n)$ for all n, it's always more <u>efficient</u> to use $h_2(n)$.
  - h_2 is more realistic than h_1 *(more informed)*, though both are optimistic.

Traditional informed search strategies

- Greedy Best first search
  - "Always chooses the successor node with the best *f* value" where *f(n) = h(n)*
  - We choose the one that is nearest to the final state among all possible choices

- A* search
  - Best first search using an "admissible" heuristic function *f* that takes into account the current cost *g*
  - Always returns the optimal solution path



Informed Search Strategies

Best First Search

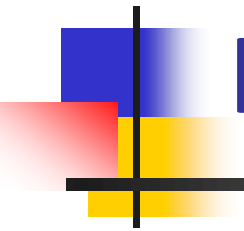
An implementation of Best First Search

function BEST-FIRST-SEARCH (*problem*, *eval-fn*)

returns a solution sequence, or failure

queuing-fn = a function that sorts nodes by *eval-fn*

return GENERIC-SEARCH (*problem*,*queuing-fn*)

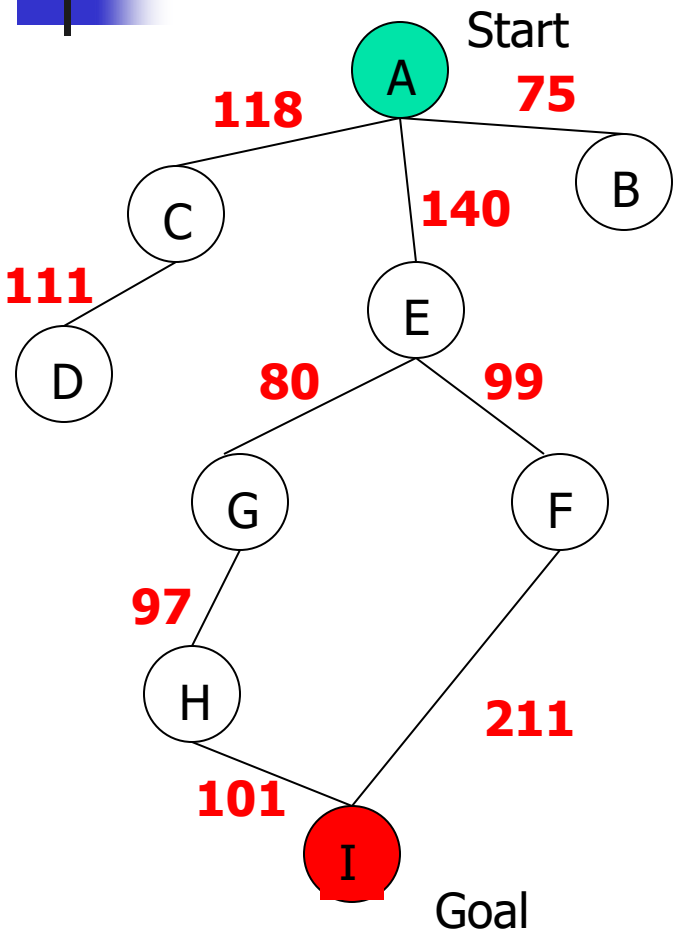


Informed Search Strategies

Greedy Search

eval-fn: f(n) = h(n)

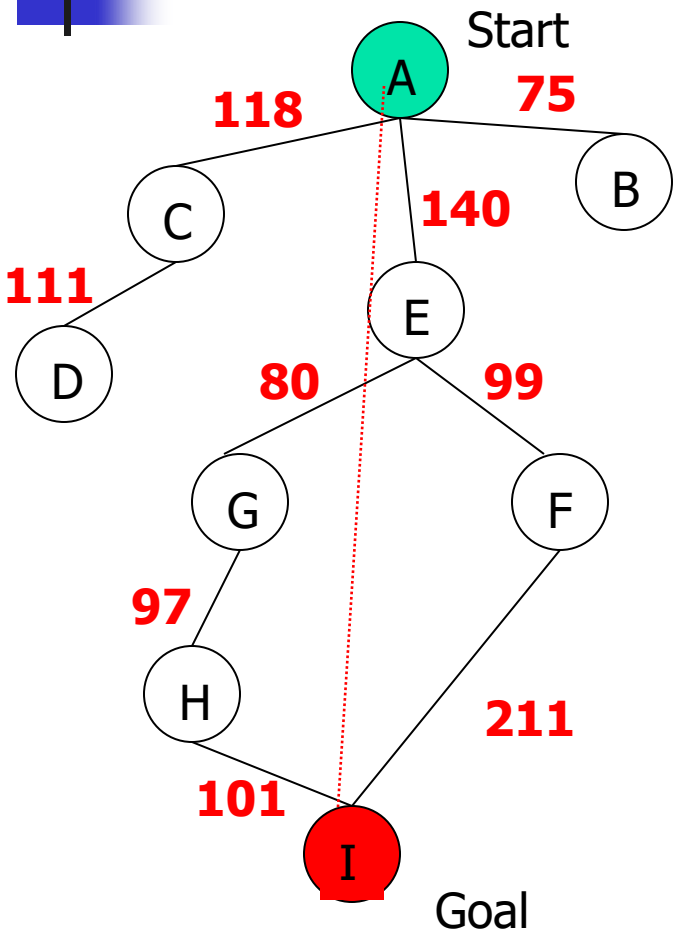
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

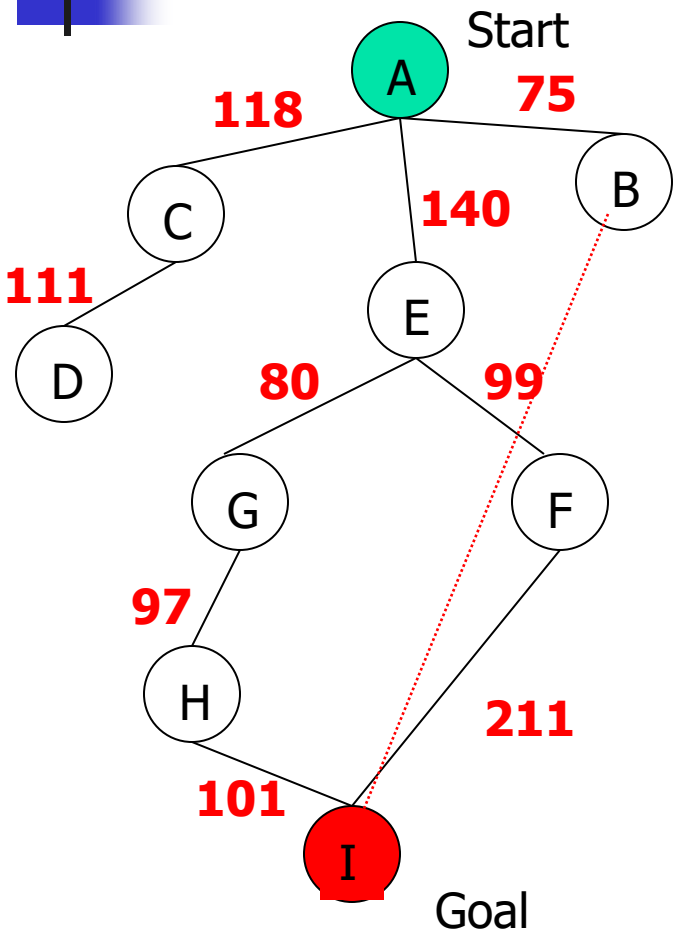
Greedy Search



State	Heuristic: h(n)
A	**366**
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

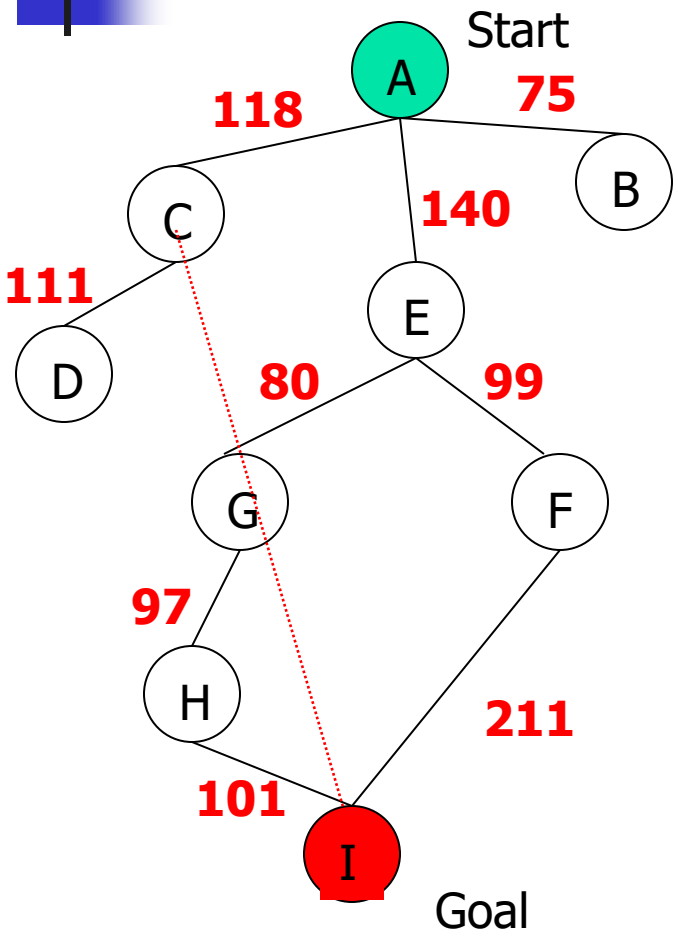
Greedy Search



State	Heuristic: h(n)
A	366
B	**374**
C	329
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

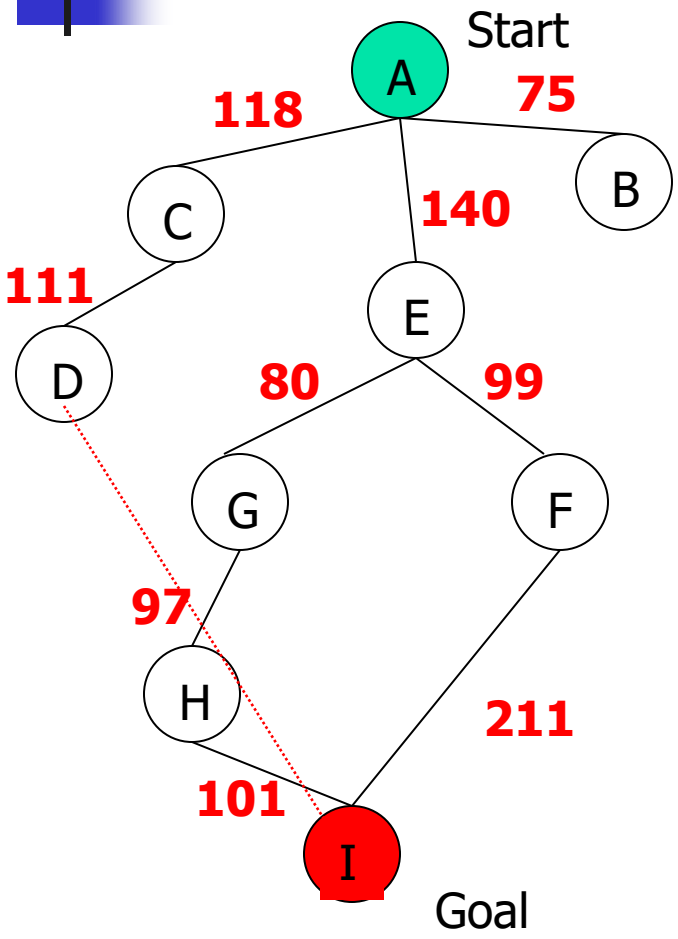
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	**329**
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

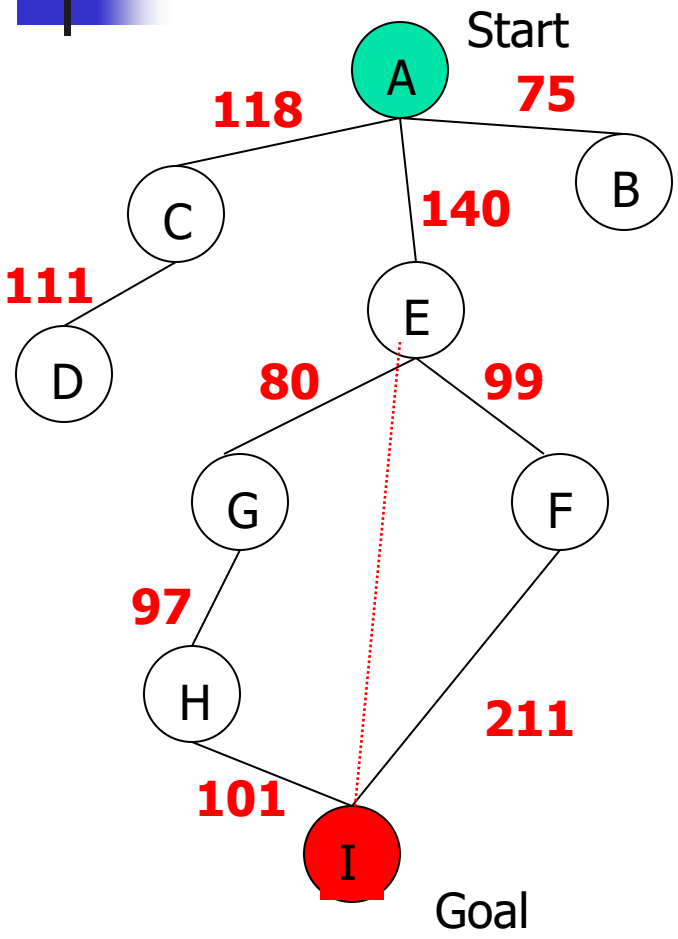
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	**244**
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

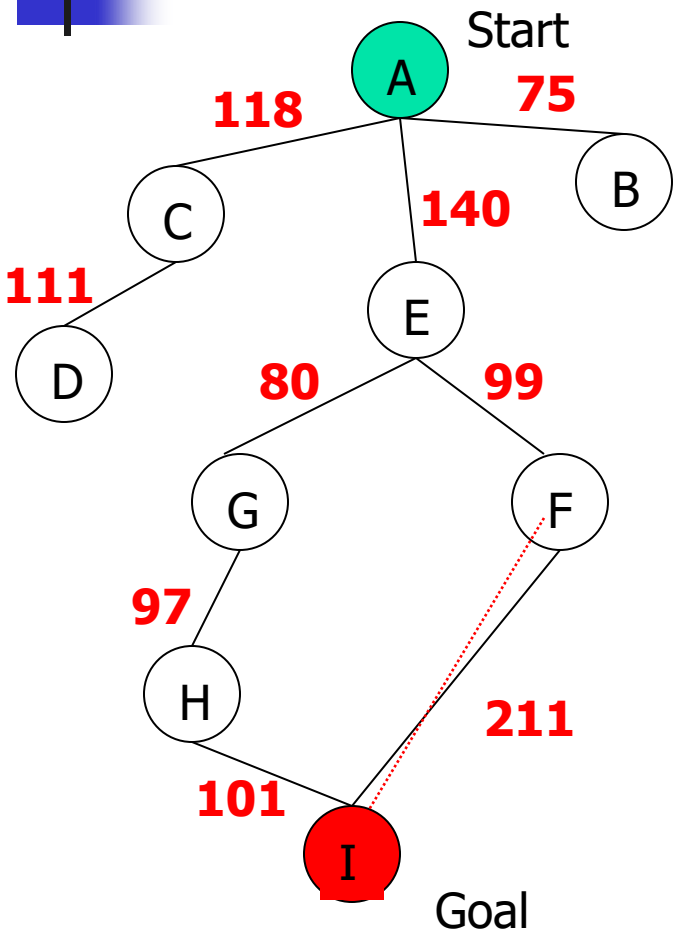
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	**253**
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

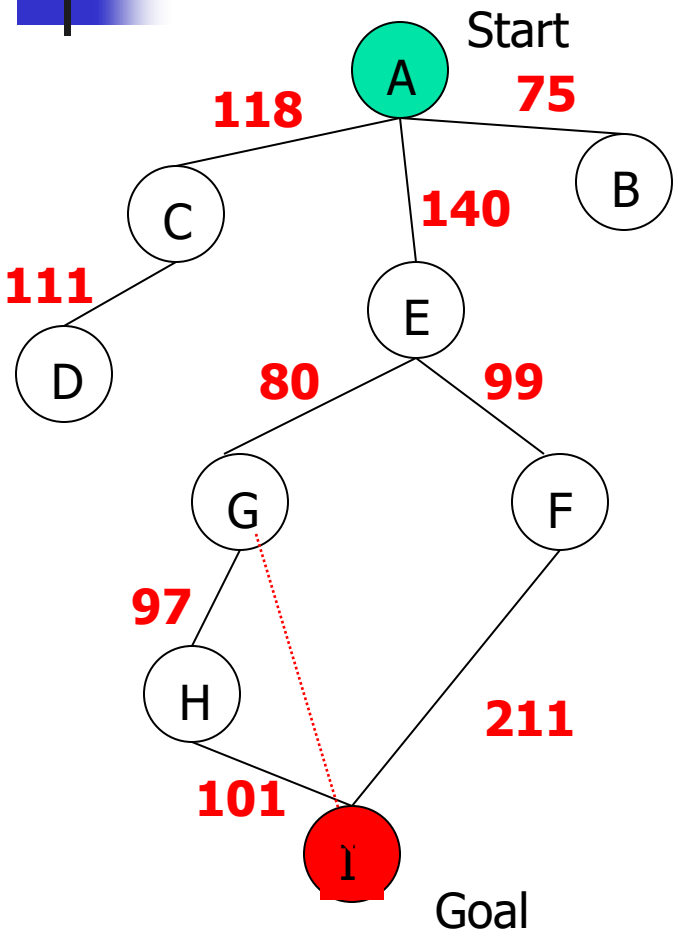
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	**178**
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

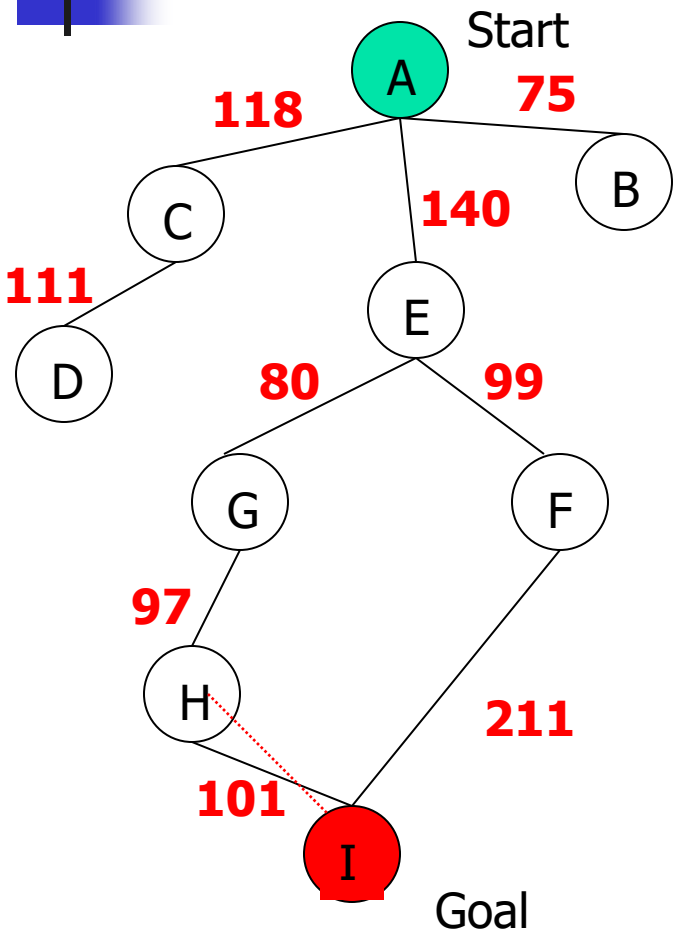
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	**193**
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

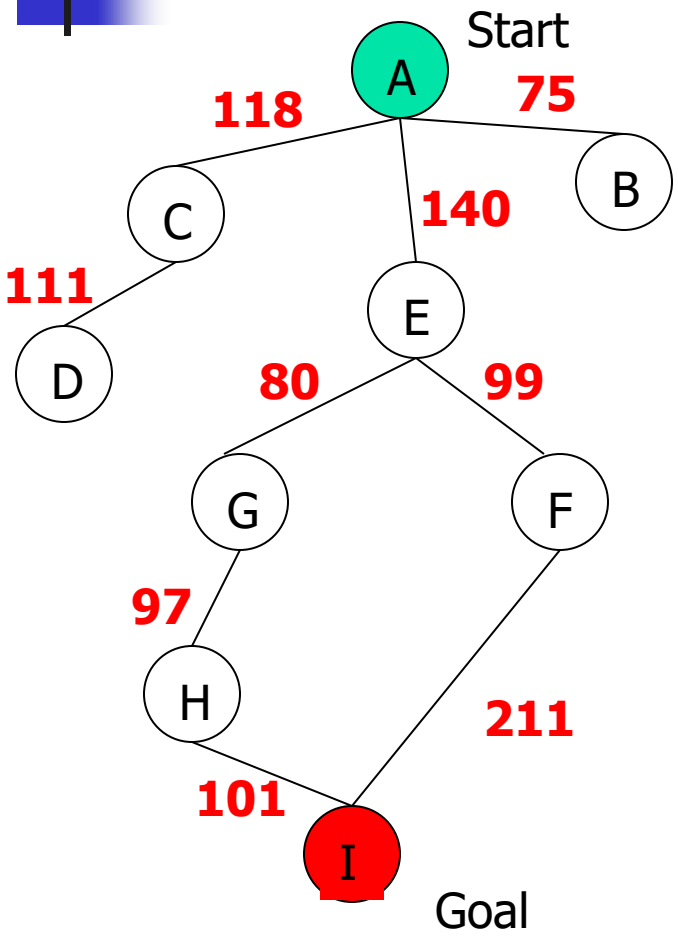
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	**98**
I	0

***f(n) = h (n)* = straight-line distance heuristic**

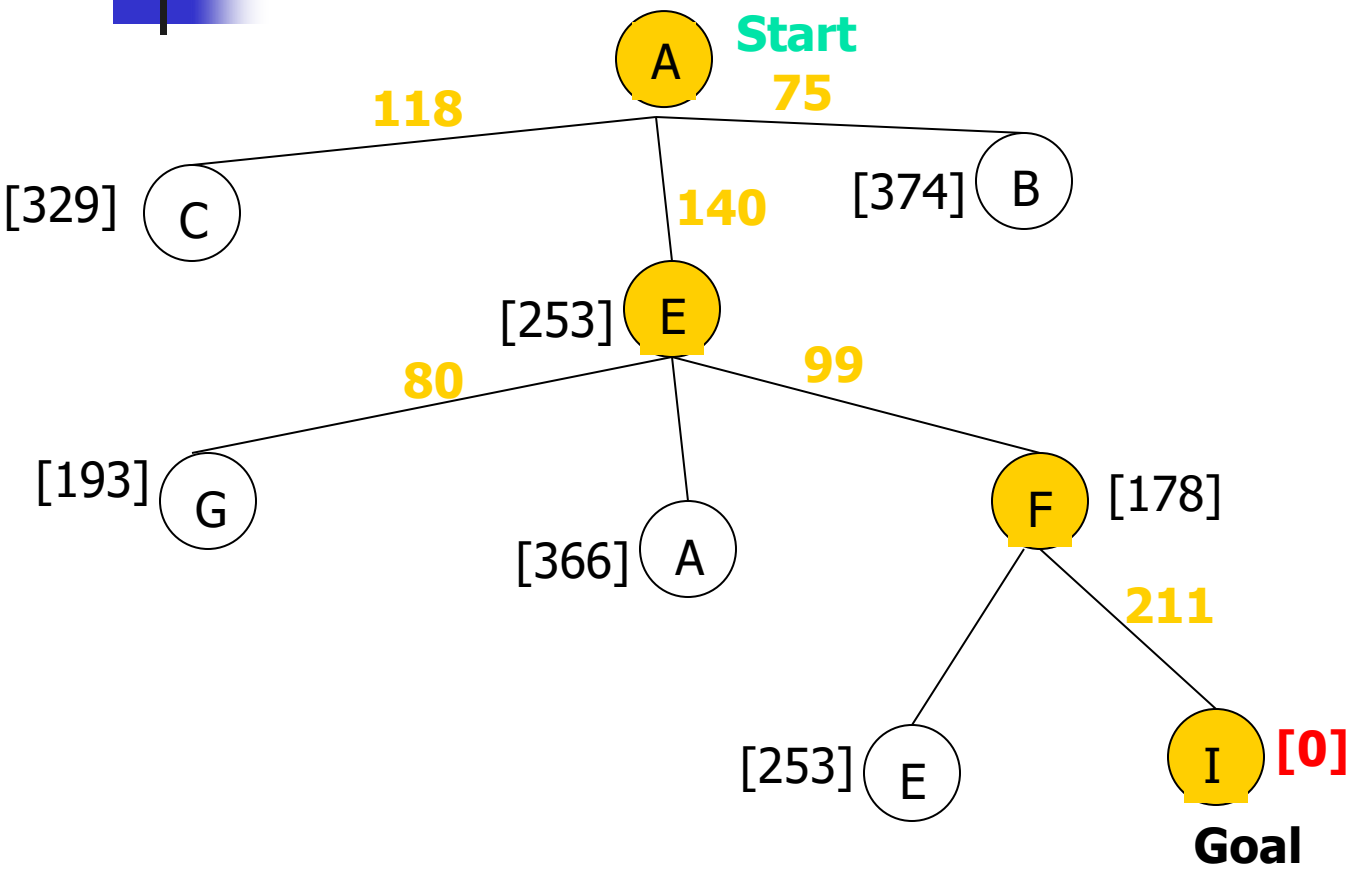
Greedy Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	**0**

***f(n) = h (n)* = straight-line distance heuristic**

Greedy Search: Tree Search

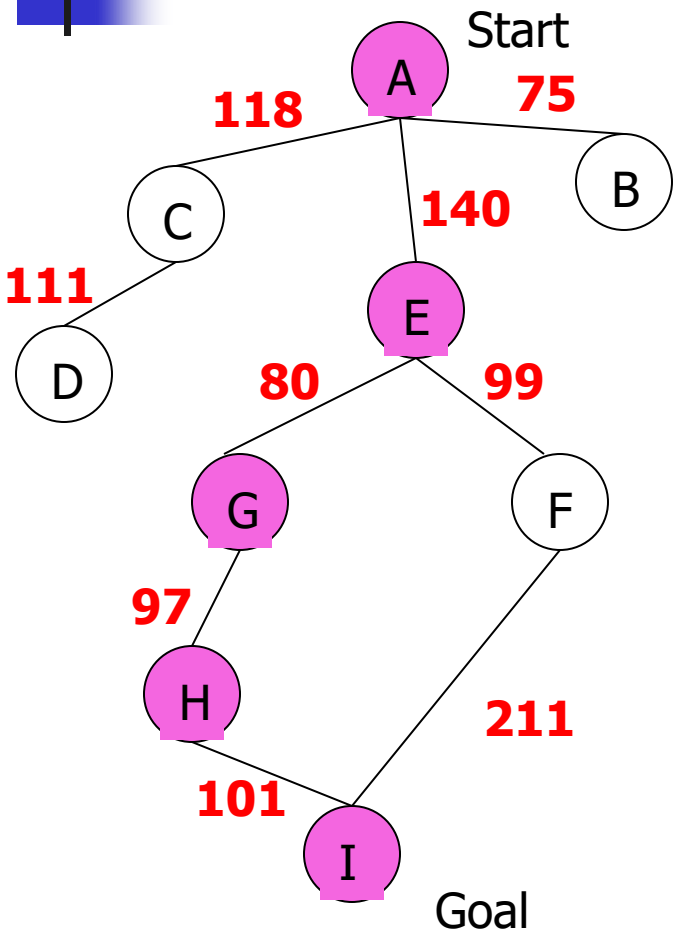


State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

Path cost(A-E-F-I) = 253 + 178 + 0 = 431

dist(A-E-F-I) = 140 + 99 + 211 = 450

Greedy Search: Optimal ?

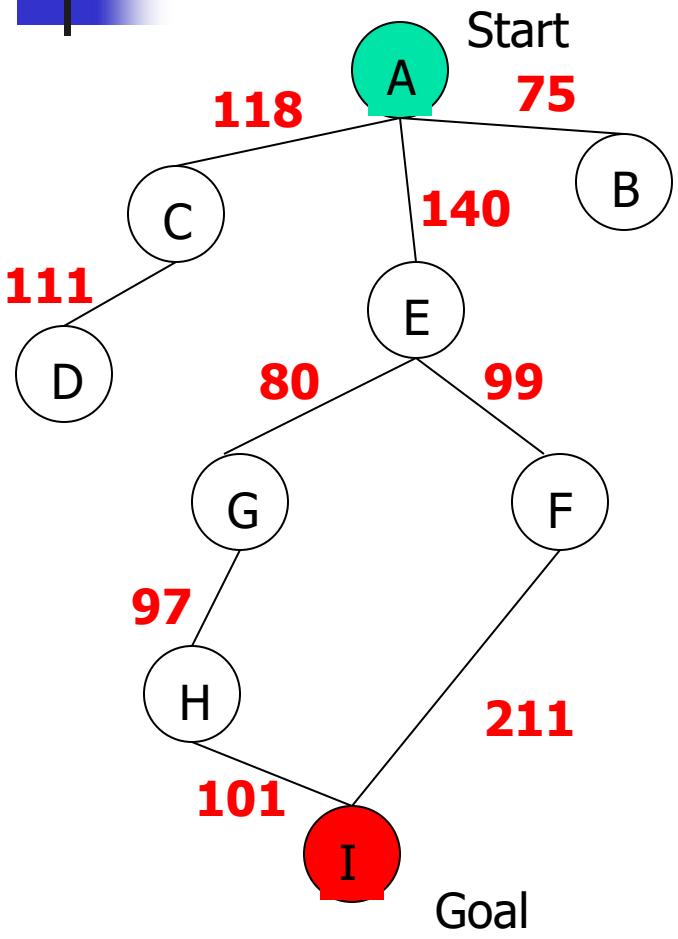


State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

dist(A-E-G-H-I) =140+80+97+101=418

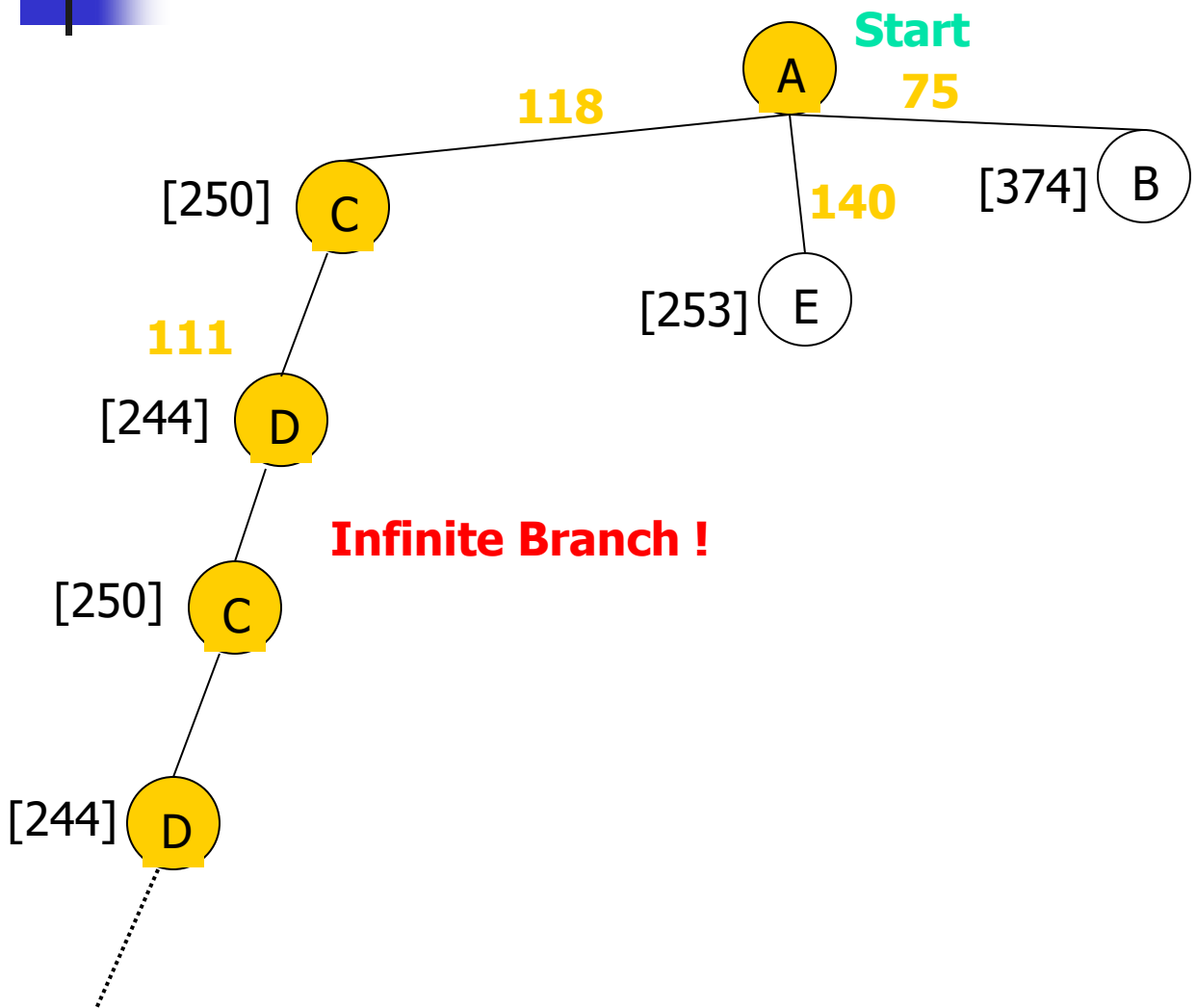
Greedy Search: Complete ?



State	Heuristic: h(n)
A	366
B	374
** C	**250**
D	244
E	253
F	178
G	193
H	98
I	0

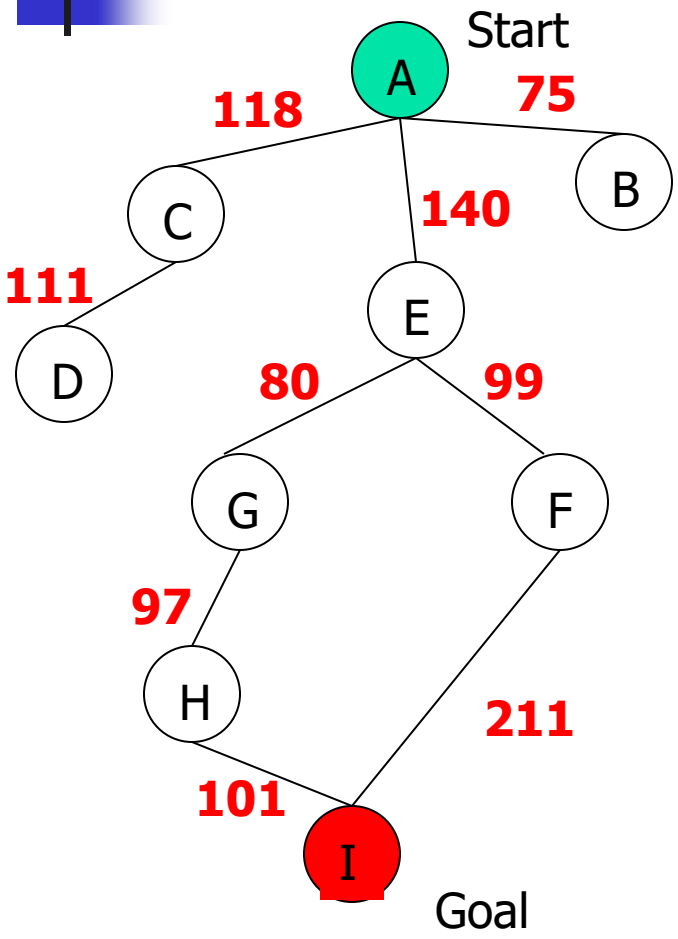
***f(n) = h (n)* = straight-line distance heuristic**

Greedy Search: Tree Search



State	Heuristic: h(n)
A	366
B	374
C	250
D	244
E	253
F	178
G	193
H	98
I	0

Greedy Search: Time and Space Complexity ?



- Greedy search is not optimal.
- Greedy search is incomplete without systematic checking of repeated states.
- In the worst case, the Time and Space Complexity of Greedy Search are both $O(b^m)$

Where b is the branching factor and m the maximum path length